

Transpiling Quantum Circuits on Graph Databases

Valter Johan Edvard Uotila

Aalto University

Espoo, Finland

University of Helsinki

Helsinki, Finland

valter.uotila@aalto.fi

Keywords

circuit transpilation, circuit compilation, graph databases, ZX-calculus

ACM Reference Format:

Valter Johan Edvard Uotila. 2025. Transpiling Quantum Circuits on Graph Databases. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25)*, June 22–27, 2025, Berlin, Germany. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3736393.3736694>

1 Introduction and background

Quantum computing is increasingly being applied to problems in data management [14, 16]. On the other hand, databases supporting quantum computing pipelines have not been studied as widely. Previous works have suggested using databases as scalable platforms for quantum simulation [4, 13], and promising systems exist for compiling circuits on relational databases [9, 10]. Quantum computers will likely generate massive amounts of data at every level of quantum circuit execution, and databases will likely play a key role in hybrid quantum-classical software stacks. This abstract outlines a framework for performing quantum circuit transpilation on graph databases and the expected performance and usability improvements.

Quantum circuit transpiling and compilation is the phase where the user-defined quantum circuit is translated into a circuit that can be efficiently executed on real hardware. This is a complex optimization problem with multiple steps and levels, which include mapping circuit qubits to the hardware qubits [6], rewriting the circuit with hardware-specific gates [10], and optimizing gates to obtain a shorter circuit [8]. For example, Qiskit transpiler [5] and Tket compiler [12] include the following steps:

- (1) Initial circuit analysis and representation
- (2) Circuit optimization in terms of logical qubits
- (3) Assign logical qubits to physical qubits
- (4) Convert to the target hardware's native gate set
- (5) Perform noise-aware optimizations
- (6) Generate executable code for the target quantum hardware

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

Q-Data '25, Berlin, Germany

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1944-8/25/06

<https://doi.org/10.1145/3736393.3736694>

2 Methods and demonstration

Most previous rewriting steps can be implemented as transformations on graphs [3]. In this formalism, quantum circuits are viewed as graphs using a novel mathematical framework called ZX-calculus [15]. ZX-calculus not only expresses circuits as graphs but also formalizes the compilation steps with a specific graphical calculus. We store the ZX-graphs in a graph database and implement the compilation and optimization routines as graph queries.

We will connect the theory and practice through a category-theoretical construction called a *span* [2], which has already been implemented to modify graphs on graph databases [11]. We propose it as a key link between ZX-calculus rewrite rules, graph rewrites in practice, and graph database queries. Span constructions are mapped to graph queries, which enable node cloning, removal of nodes or edges, node merging, and addition of nodes or edges. The final optimized graph is mapped back to a quantum circuit. Figure 1 presents the pipeline summary.

Next, we describe a simple demonstration to show the feasibility of the proposed method. We choose the 100-qubit Quantum Volume circuit from QASMBench [7]. The depth of the circuit is 700, and it has 40,000 U3-gates and 15,000 CNOT-gates, which makes 55,000 gates in total. Since U3-gates are not usually basic or hardware-supported gates, the corresponding ZX-diagram has exactly 230,000 Pauli Z and X operations. The transformation from the QASM file into a ZX-graph and its storage in the MemGraph graph database with the baseline implementations takes about 25 seconds.

Then, we perform the spider fusion operation for this graph. Spider fusion means that the graph nodes with the same color can be combined and their phases are summed, as demonstrated in Figure 2. This fusion process can be expressed as a Cypher query, which is outlined in Figure 3. Instead of performing fusion between two nodes, the graph query naturally allows us to perform three-node fusion simultaneously, which is two two-node fusions in sequence. The total execution time of this query is about 1.5 seconds. After executing the Cypher query, the number of total operations becomes 200,000, demonstrating that 30,000 gate operations were removed.

3 Expected improvements and limitations

Executing compilation steps as graph rewrites on graph databases will bring multiple performance and usability benefits, especially for large and deep circuits. First, graph databases can be easily employed to perform circuit analysis and compute statistics. Pattern matching and graph rewriting are optimized and utilize indexing. Graph query languages are especially designed for graph rewriting. Graph databases can store more than gate names and types: they can capture previously used gate parameters, qubit allocations, and

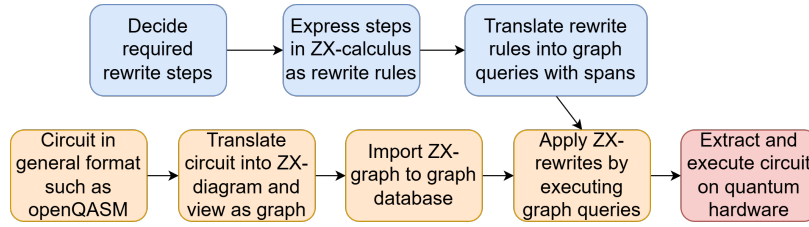


Figure 1: Full pipeline

$$n \text{ --- } (\alpha \text{ --- } \beta) \text{ --- } m \mapsto n \text{ --- } (\alpha + \beta) \text{ --- } m$$

Figure 2: Spider fusion is a basic ZX-calculus rewrite rule

```
// Step 1: Match the pattern
MATCH (n)-[k]->(m)-[h]->(l)
WHERE n.type = "Z" AND m.type = "Z"
      AND l.type = "Z" AND k.t = "simple"
      AND h.type = "simple"
// Step 2: Sum phases
WITH n.phase + m.phase + l.phase AS
      combined_phase
// Step 3: Create the new merged node
CREATE (combined:Node)
SET combined.phase = combined_phase
// Step 4: Redirect edges (lengthy/technical,
      omitted)
// Step 5: Delete old nodes and edges
WITH old_nodes
UNWIND old_nodes AS old
DETACH DELETE old
```

Figure 3: Simplified Cypher query to perform three-node spider fusion

error models. This creates the possibility of considering circuits as evolving objects in hybrid quantum-classical pipelines.

Applied compilation routines can be rolled back, and the same circuit can have multiple efficiently stored versions. As quantum circuits grow, their processing may eventually exceed memory limitations, which motivates using on-disk processing techniques. In multi-user quantum computing platforms, ACID properties will become critical. Quantum computers can be modeled as graphs in the same graph database, and qubit mapping can be optimized at the database level.

Although ZX-calculus and graph databases integrate seamlessly at the graph theoretical level, multiple technical challenges exist. One key challenge is that rewriting circuits with ZX-calculus does not necessarily ensure that they remain valid, i.e., they become proper graphs that only represent the underlying tensor network. Thus, the circuits have to be extracted, and there are methods for extraction [1]. Unfortunately, these methods require sequential graph processing, which might require multiple query executions in sequence.

4 Conclusion

This abstract outlined an idea for storing quantum circuits in graph databases and executing circuit optimization routines as graph queries. We discussed the expected improvements and identified possible challenges. The growing quantum computing research and industry will introduce a new class of complex data management challenges for database researchers and practitioners, and transpiling quantum circuits serves as an excellent example.

References

- [1] Miriam Backens, Hector Miller-Bakewell, Giovanni de Felice, Leo Lobski, and John van de Wetering. 2021. There and back again: A circuit extraction tale. (March 2021). doi:10.22331/q-2021-03-25-421
- [2] Andrea Corradini, Tobias Heindel, Frank Hermann, and Barbara König. 2006. Sesqui-Pushout Rewriting. In *Graph Transformations*, Andrea Corradini, Hartmut Ehrig, Ugo Montanari, Leila Ribeiro, and Grzegorz Rozenberg (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 30–45.
- [3] Ross Duncan, Aleks Kissinger, Simon Perdrix, and John van de Wetering. 2020. Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus. *Quantum* 4 (June 2020), 279. doi:10.22331/q-2020-06-04-279
- [4] Rihan Hai, Shih-Han Hung, Tim Coopmans, Tim Littau, and Floris Geerts. 2025. Quantum Data Management in the NISQ Era: Extended Version. arXiv:2409.14111 [quant-ph] <https://arxiv.org/abs/2409.14111>
- [5] IBM Quantum. 2024. Qiskit Transpiler API Documentation. <https://docs.quantum.ibm.com/api/qiskit/transpiler>
- [6] Aleks Kissinger and Arianne Meijer van de Griend. 2019. CNOT circuit extraction for topologically-constrained quantum memories. arXiv:1904.00633 [quant-ph] <https://arxiv.org/abs/1904.00633>
- [7] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2022. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Transactions on Quantum Computing* (2022).
- [8] Arianne Meijer-van de Griend and Ross Duncan. 2023. Architecture-Aware Synthesis of Phase Polynomials for NISQ Devices. *Electronic Proceedings in Theoretical Computer Science* 394 (Nov. 2023), 116–140. doi:10.4204/eptcs.394.8
- [9] Ioana Moflic. 2024. Scalable Compilation and Equivalence Checking of Quantum Circuits – with an Application to Quantum Error Correction. (Sept. 2024). <https://aaltodoc.aalto.fi/handle/123456789/131707>
- [10] Ioana Moflic. 2025. Pandora: Ultra-Large-Scale Quantum Circuit Design Automation. <https://github.com/ioanamoflic/pandora>
- [11] Eugenia Oshurko. 2024. ReGraph: A Graph Programming Library. <https://dev.executableknowledge.org/ReGraph/index.html>
- [12] Quantinuum. 2024. TKET User Manual: Compilation Overview. https://docs.quantinuum.com/tket/user-guide/manual/manual_compiler.html
- [13] Immanuel Trummer. 2024. Towards Out-of-Core Simulators for Quantum Computing. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications, Q-Data 2024, Santiago, Chile, June 9–15, 2024*, Ibrahim Sabek, Immanuel Trummer, and Stefan Prestel (Eds.). ACM, doi:10.1145/3665225.3665441
- [14] Valter Uotila. 2022. Synergy between Quantum Computers and Databases. In *PhD Workshop of the 48th International Conference on Very Large Databases (VLDB 2022)*.
- [15] John van de Wetering. 2020. ZX-calculus for the working quantum computer scientist. arXiv:2012.13966 (Dec. 2020). doi:10.48550/arXiv.2012.13966
- [16] Tobias Winker, Sven Groppe, Valter Uotila, Zhengtong Yan, Jiaheng Lu, Maja Franz, and Wolfgang Mauerer. 2023. Quantum Machine Learning: Foundation, New Techniques, and Opportunities for Database Research. In *Companion of the 2023 International Conference on Management of Data (Seattle, WA, USA) (SIGMOD '23)*. Association for Computing Machinery, New York, NY, USA, 45–52. doi:10.1145/3555041.3589404